

MINERÍA DE DATOS

Maximiliano Ojeda

muojeda@uc.cl



IIC-2433



Series de Tiempo

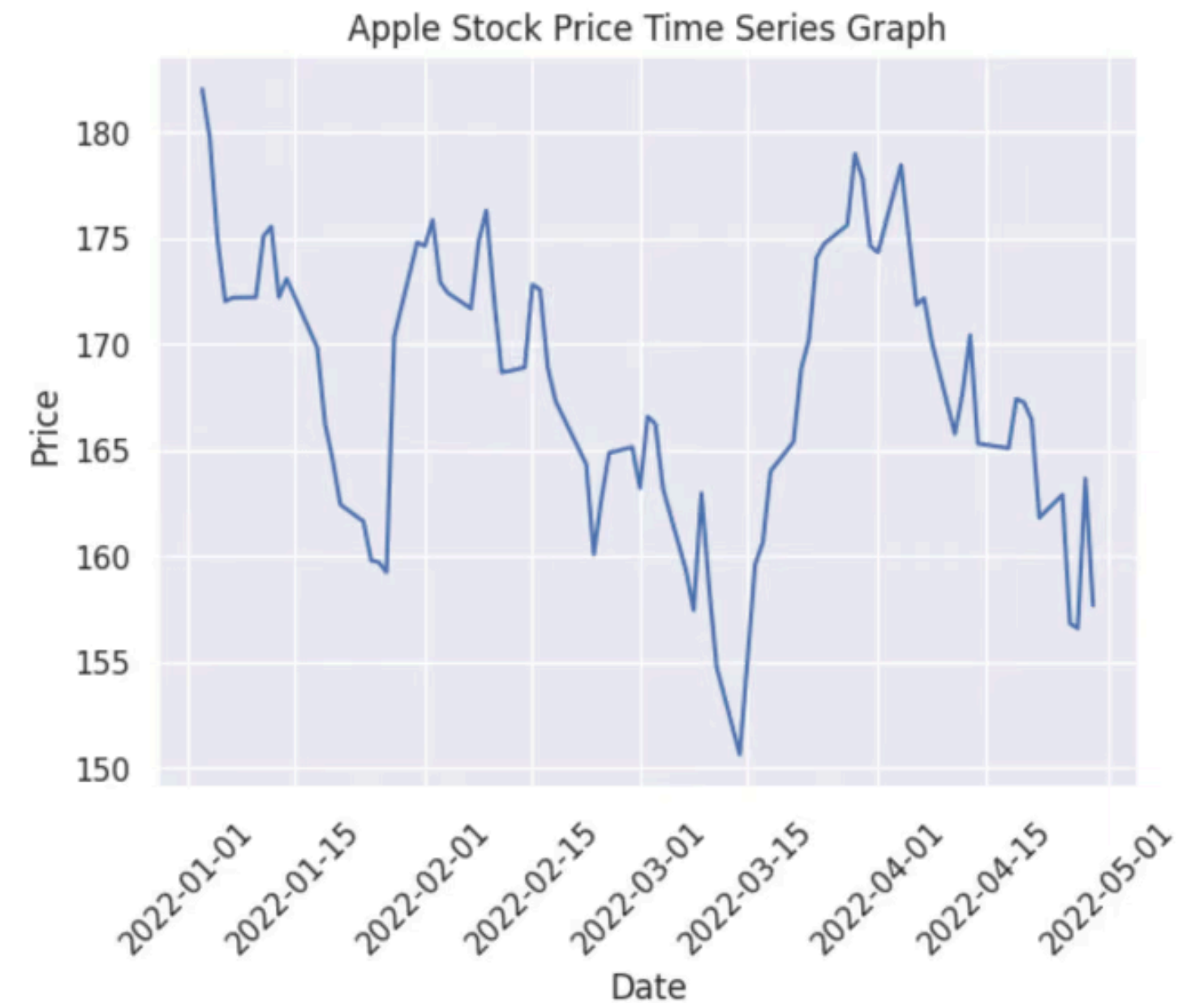
¿Qué es una serie de tiempo?

Una serie de tiempo es una secuencia de observaciones de una misma variable, medidas en instantes sucesivos y ordenados en el tiempo (por ejemplo, cada hora, cada día, cada mes).

El **orden importa y las observaciones no son independientes entre sí**. Esto rompe el supuesto de "datos i.i.d." que subyace a la mayoría de los modelos clásicos de ML

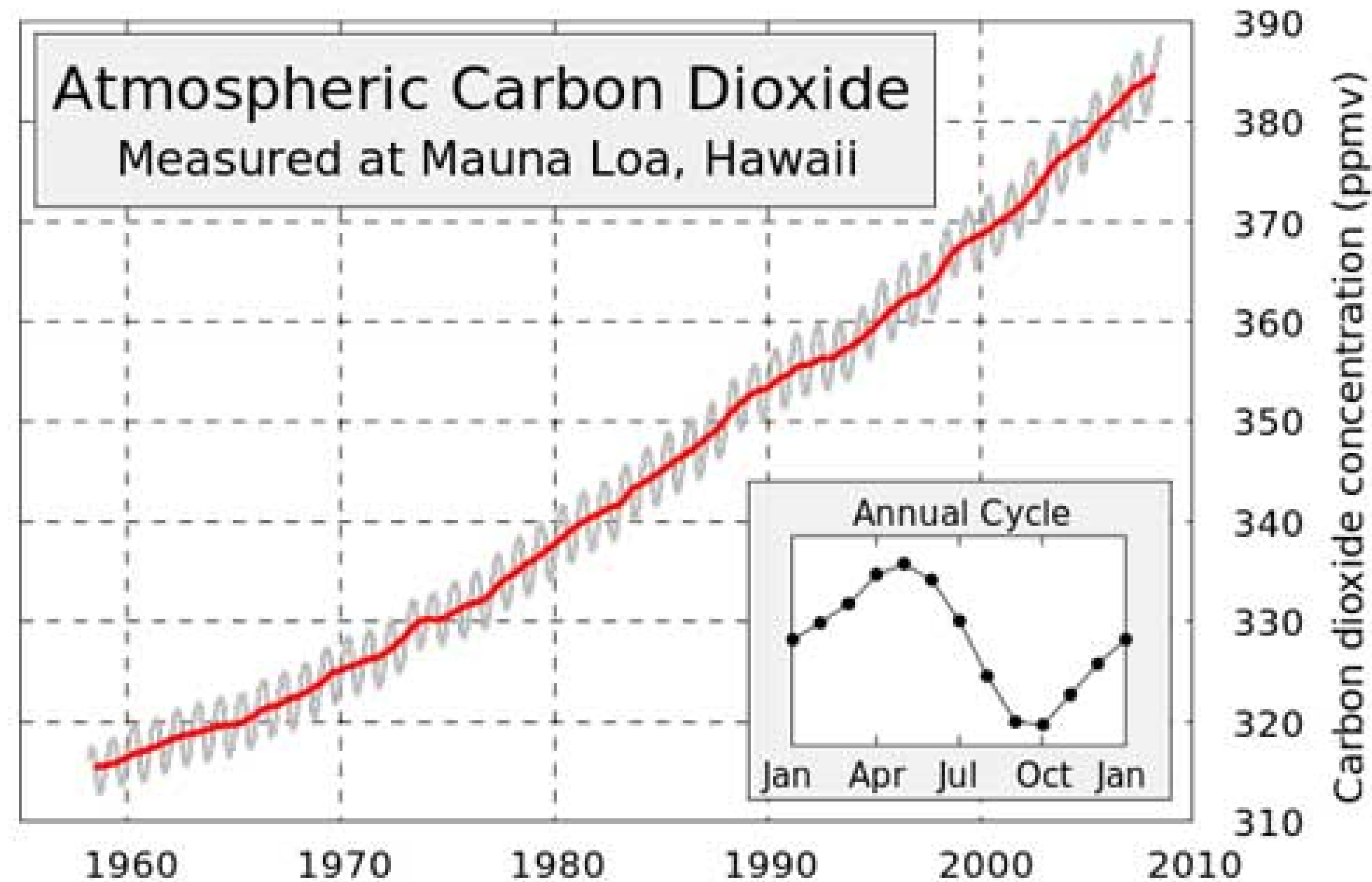
Ejemplos:

- El precio de una acción, minuto a minuto 📈
- La temperatura registrada por un sensor cada hora 🌡️
- Las ventas mensuales de un producto 💰
- Los streams diarios de una canción en Spotify 🎵



Ejemplo CO₂ atmosférico

Desde 1958, Charles David Keeling midió la concentración de CO₂ en Mauna Loa, Hawái, casi ininterrumpidamente. Muestra una tendencia ascendente clarísima con una estacionalidad anual perfecta (el CO₂ baja en verano boreal porque las plantas absorben más, sube en invierno)



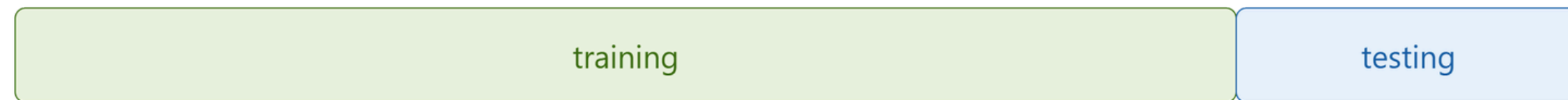
¿Qué es una serie de tiempo?

Formalmente, se puede pensar como una función x_t donde t es el índice temporal (discreto, generalmente):

$$\{x_1, x_2, x_3, \dots, x_T\}$$

- No se puede barajar los datos ni hacer un split aleatorio train/test (eso sería fuga de información del futuro hacia el pasado)
- Hay patrones estructurales que se pueden y deben modelar explícitamente: **tendencia, estacionalidad, autocorrelación**

✓ Split secuencial



✗ Split aleatorio





Componentes

Componentes Serie de Tiempo

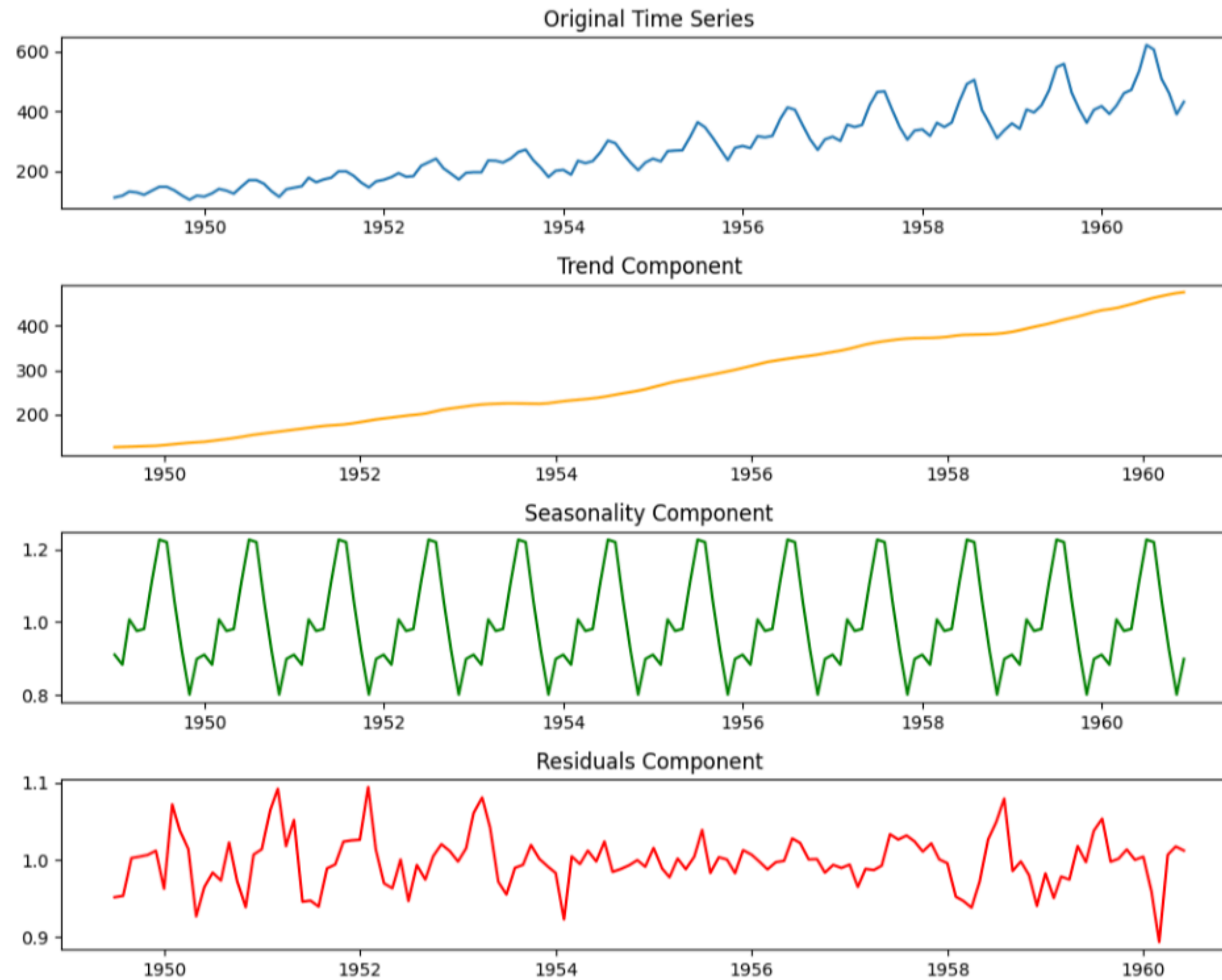
Tendencia (trend): la dirección general de largo plazo. La serie sube, baja o se mantiene estable a lo largo de mucho tiempo, ignorando fluctuaciones de corto plazo.

Estacionalidad (seasonality): patrones que se repiten en **intervalos fijos y conocidos** (diario, semanal, mensual, anual). Por ejemplo, ventas de retail que suben cada diciembre, o tráfico web que baja los fines de semana.

Ciclo (cyclic): fluctuaciones también repetitivas, pero **sin un periodo fijo** y generalmente de mayor duración que la estacionalidad. El ejemplo clásico son los ciclos económicos (expansión/recesión), que duran años pero no un número exacto de meses.

Ruido (noise/residual): lo que queda después de remover tendencia, estacionalidad y ciclo. **Variación aleatoria que el modelo no puede explicar.**

Componentes Serie de Tiempo



Componentes Serie de Tiempo

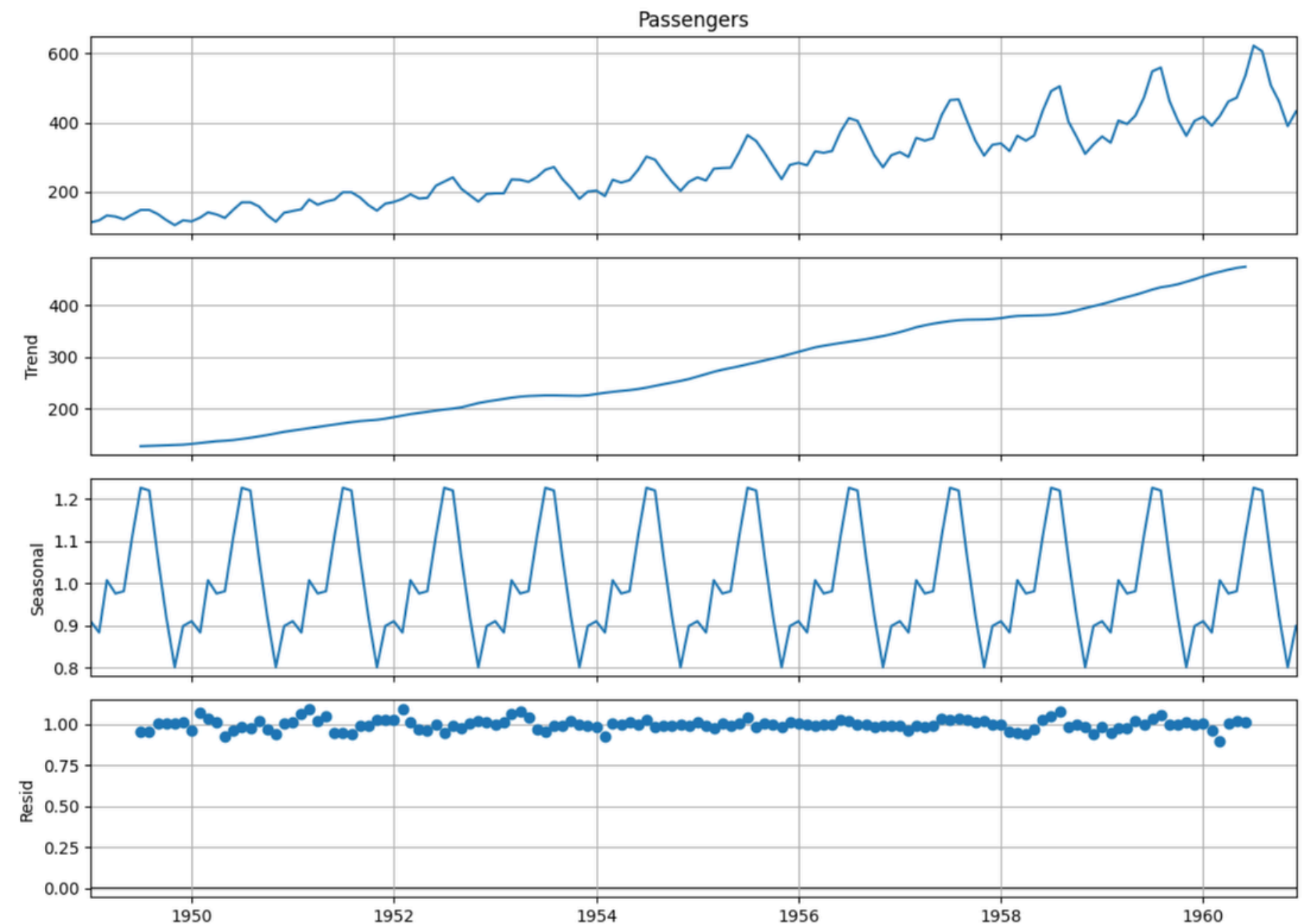
```
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose
```

```
url = "airline-passengers.csv"
df = pd.read_csv(url)
df["Month"] = pd.to_datetime(df["Month"])
df = df.set_index("Month")
df = df.asfreq("MS")
df.head()
```

```
resultado = seasonal_decompose(df["Passengers"],
                                model="multiplicative", period=12)
```

```
fig = resultado.plot()
fig.set_size_inches(11, 8)
plt.show()
```

Passengers	
Month	
1949-01-01	112
1949-02-01	118
1949-03-01	132
1949-04-01	129
1949-05-01	121



Estacionariedad

Una serie de tiempo es estacionaria cuando **sus propiedades estadísticas no cambian en el tiempo**. Formalmente (estacionariedad débil o de segundo orden), se requiere que:

- Media constante: $E[y_t] = \mu$ para todo t (no hay tendencia)
- Varianza constante: $Var(y_t) = \sigma^2$ para todo t (no hay "abanico" que crece o se achica)
- Autocovarianza que depende solo del rezago, no del tiempo: $Cov(y_t, y_{t+k})$ depende de k , pero no de en qué momento tiempo estamos

Por qué importa

Los modelos clásicos (ARIMA) están contruidos matemáticamente asumiendo estacionariedad. Algunas razones:

- La inferencia estadística (intervalos de confianza, tests de significancia) que usan estos modelos depende de propiedades asintóticas que solo se cumplen bajo estacionariedad.
- Series no estacionarias tienden a producir **regresiones espurias**

Por eso el flujo típico de trabajo es: **verificar estacionariedad** → si no la hay, **transformar la serie** → recién ahí **ajustar el modelo clásico**.

Test de Dickey-Fuller Aumentado (ADF)

Es el test más usado para verificar estacionariedad de forma cuantitativa (en lugar de mirar el gráfico).

Hipótesis:

- H_0 : la serie no es estacionaria
- H_1 : la serie es estacionaria

Regla de decisión: si el p-value < 0.05, rechazamos H_0 y concluimos que la serie es estacionaria.

```
from statsmodels.tsa.stattools import adfuller

resultado = adfuller(serie.dropna())
print(f"Estadístico ADF: {resultado[0]:.4f}")
print(f"p-value: {resultado[1]:.4f}")
# p-value < 0.05 -> estacionaria
```

Transformaciones para estacionariedad

1. Log-transform

Se usa cuando la **varianza no es constante**. Típicamente cuando la amplitud de las fluctuaciones crece junto con el nivel de la serie

$$y'_t = \log(y_t)$$

Estabiliza la varianza al comprimir proporcionalmente los valores grandes más que los pequeños.

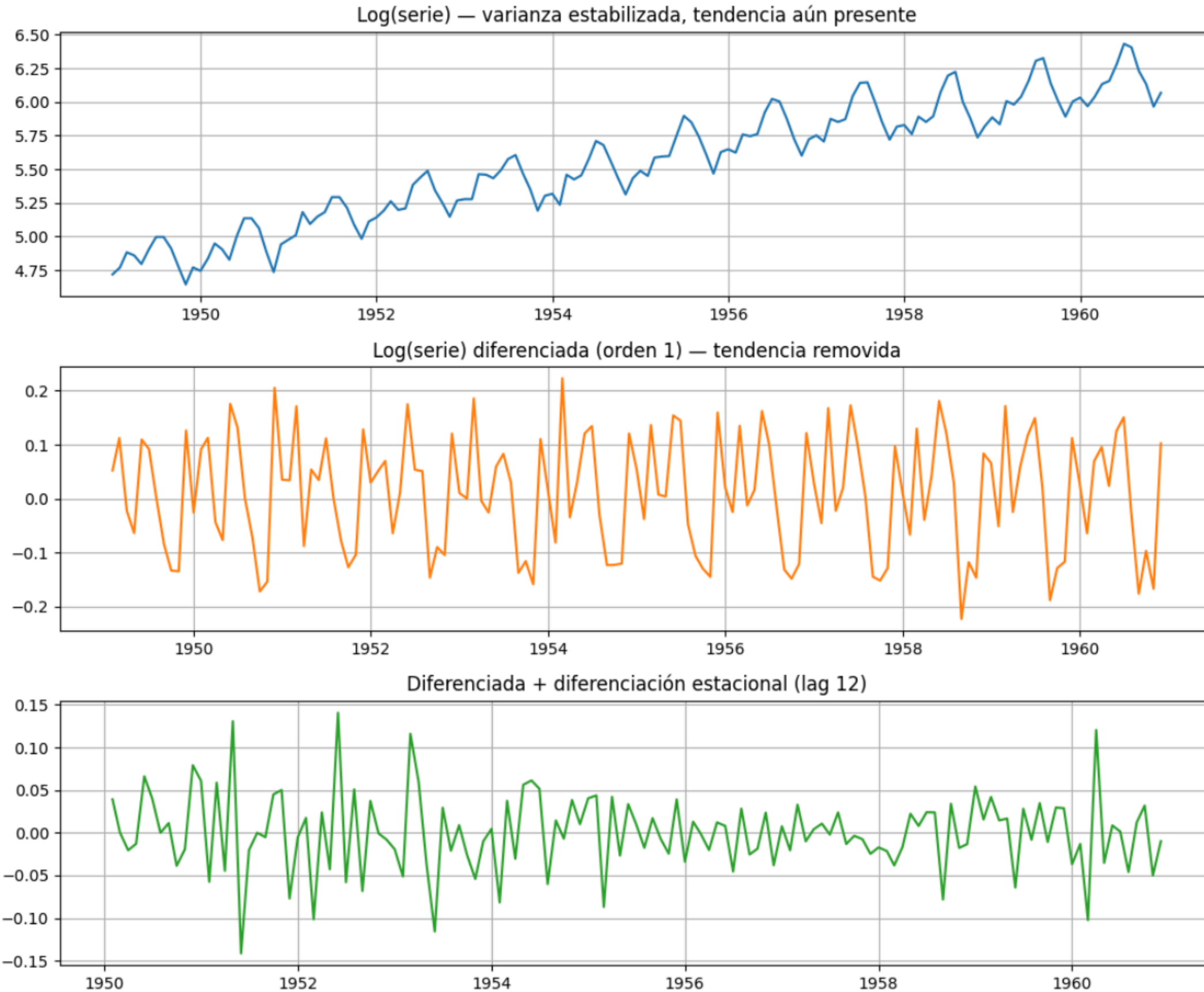
2. Diferenciación

Se usa cuando hay **tendencia (media no constante)**. Modelamos el cambio entre observaciones consecutivas:

$$\Delta y_t = y_t - y_{t-1}$$

Si con una diferenciación no basta (todavía hay tendencia residual), se puede diferenciar de nuevo. *En la práctica casi nunca se necesita más de 1 o 2 diferenciaciones*

Transformaciones para estacionariedad



Autocorrelación

La autocorrelación mide **qué tan relacionado está un valor de la serie con sus propios valores pasados**

ACF (Autocorrelation Function)

Mide la correlación entre y_t y y_{t-k} para distintos rezagos k (lag 1, lag 2, lag 3, ...), sin controlar por los rezagos intermedios:

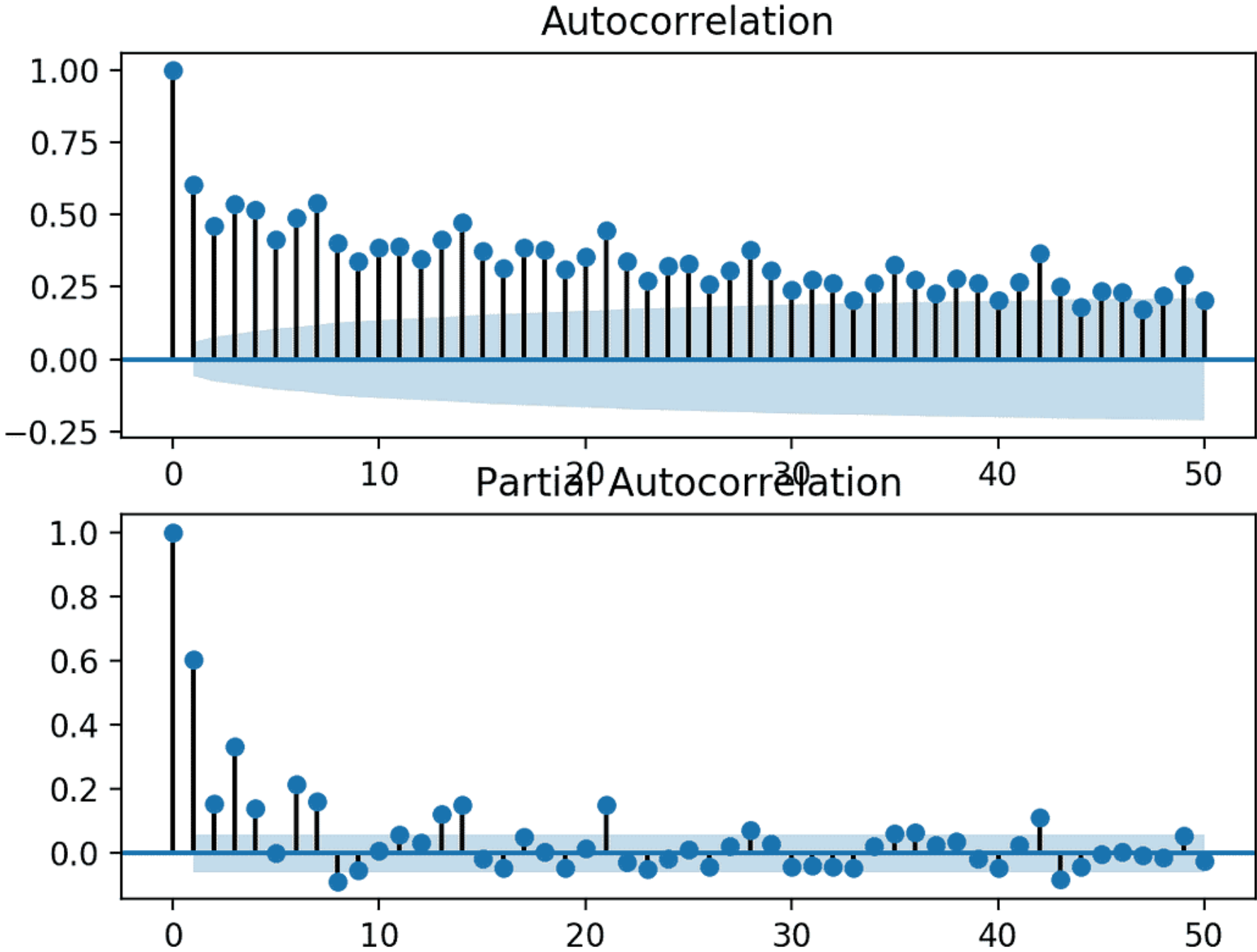
$$\rho_k = \frac{Cov(y_t, y_{t-k})}{Var(y_t)}$$

Si hay correlación en el lag 2, parte de esa correlación puede venir "prestada" indirectamente a través del lag 1. La ACF no distingue eso, mezcla el efecto directo e indirecto

PACF (Partial Autocorrelation Function)

Mide la correlación entre y_t y y_{t-k} , pero **removiendo el efecto de los rezagos intermedios**. Es la correlación "neta" o "directa" en el lag k . **La PACF pregunta:** "¿ y_{t-2} aporta información adicional sobre y_t una vez que ya conozco y_{t-1} ?"

Autocorrelación





Preprocesamiento

Resampling

Resampling significa cambiar la frecuencia temporal de la serie: agregar datos de alta frecuencia a baja frecuencia (downsampling), o el proceso inverso (upsampling)

Downsampling (más común)

Tenemos datos diarios y queremos verlos mensuales, o datos por minuto y queremos verlos por hora. Debemos decidir cómo agregar los valores dentro de cada periodo:

```
serie_mensual = serie_diaria.resample("ME").mean()    # promedio del mes
serie_mensual = serie_diaria.resample("ME").sum()      # suma del mes (útil para ventas)
serie_mensual = serie_diaria.resample("ME").last()    # último valor del mes
```

Upsampling

Datos mensuales y queremos verlos diarios. Acá no se "agrega" información real, se inventan puntos intermedios, usualmente vía interpolación:

```
serie_diaria = serie_mensual.resample("D").interpolate(method="linear")
```

Missing values

Los datos faltantes son especialmente delicados en series de tiempo porque el orden importa: no se puede simplemente eliminar la fila como en un dataset tabular común, porque se rompe la continuidad temporal (algunos métodos, como modelos ARIMA, asumen espaciado regular entre observaciones)

```
serie.fillna(method="ffill") # forward fill: repite el último valor conocido  
serie.fillna(method="bfill") # backward fill: usa el próximo valor conocido  
serie.interpolate(method="linear") # interpola linealmente entre el valor anterior y el siguiente  
serie.interpolate(method="time") # como el anterior, pero pondera según la distancia temporal real
```

Cuando usar cada uno

- **ffill** es razonable si el proceso subyacente es "de estado" (ej. un sensor que no cambió de estado)
- **interpolate** es mejor cuando se espera una transición suave entre el punto anterior y el siguiente (temperatura, precios)

Ninguna de estas dos es apropiada si hay bloques largos de datos faltantes. Ahí interpolar linealmente puede introducir tendencias falsas. En esos casos conviene marcar explícitamente esos periodos o usar modelos que manejen missing values de forma nativa

Smoothing con Medias móviles

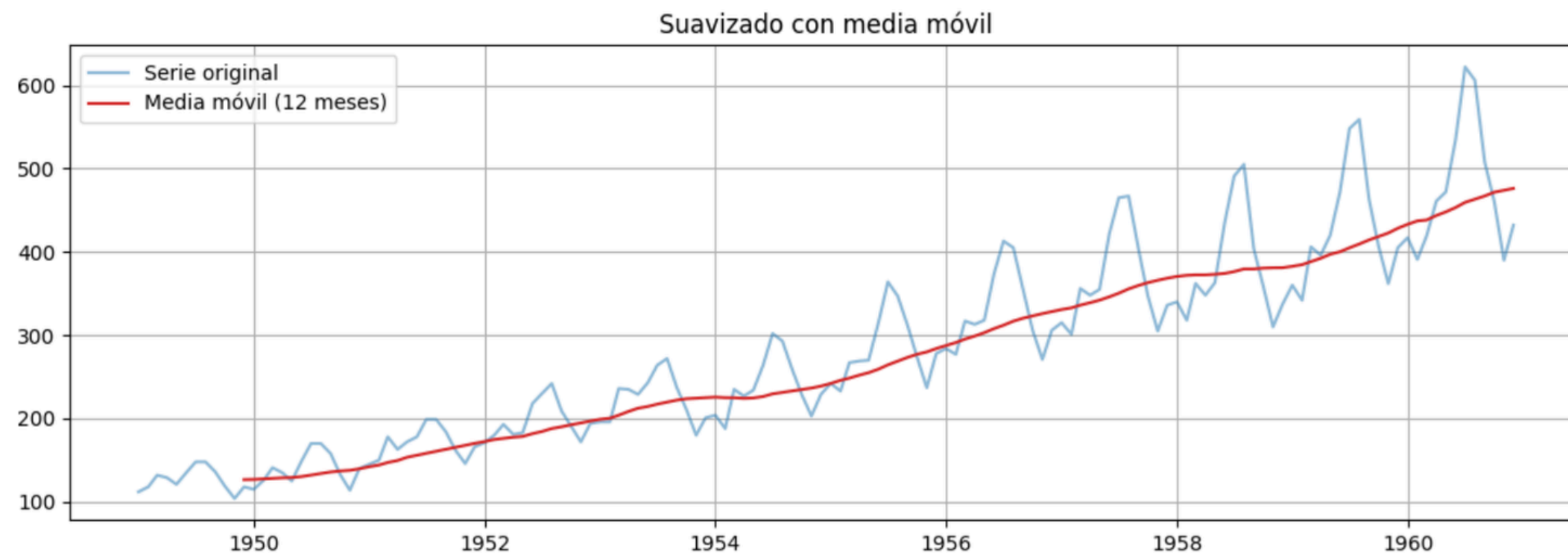
El suavizado sirve para remover ruido de corto plazo y dejar visible el patrón subyacente (tendencia, o incluso para hacer una decomposición manual simple).

```
serie.rolling(window=12).mean()
```

Esto calcula, para cada punto, el promedio de las window observaciones anteriores (incluyéndose a sí misma por defecto). Cuanto más grande la ventana, más suave el resultado, pero también más "atraso" introduce respecto a la serie original.

Variantes

```
serie.rolling(window=12, center=True).mean() # usa half-window antes y half-window después  
serie.ewm(span=12).mean() # media móvil exponencial: pondera más lo reciente
```



MINERÍA DE DATOS

Maximiliano Ojeda

muojeda@uc.cl



IIC-2433